

Sistema de Hospedagem

Relatório Técnico da Sprint 4

Evolução arquitetural
Tarifação flexível e múltiplos meios de pagamento com os padrões Strategy, Factory e Singleton.

Integrantes	Lucas Silva Pedro Henrique Dias
Disciplina	Programação Modular
Instituição	PUC Minas
Data	2 de julho de 2026

1. Funcionalidades escolhidas

A Sprint 4 conclui a evolução arquitetural do Sistema de Hospedagem. O grupo selecionou duas funcionalidades que possuem variações de regras e, por isso, se beneficiam diretamente de objetos intercambiáveis.

1.1 Opção 1 - Tarifação flexível

A diária deixa de depender de uma regra fixa. A interface `PoliticaTarifa` define o contrato comum, enquanto `TarifaPadrao`, `TarifaAltaTemporada`, `TarifaBaixaTemporada` e `TarifaPromocional` implementam ajustes independentes. O cálculo do aluguel e a previsão de diária delegam o ajuste para a política ativa.

1.2 Opção 4 - Múltiplos meios de pagamento

O sistema passa a aceitar `DINHEIRO`, `CARTAO_CREDITO`, `CARTAO_DEBITO` e `PIX`. Cada forma possui uma `Strategy` própria para validação e processamento. A `Factory` centraliza a seleção do processador, e o serviço registra o pagamento confirmado associado ao aluguel. O recibo consolida cliente, quarto, diárias, valor, forma e status.

Resultado arquitetural

As regras variáveis saem de condicionais espalhadas e passam a viver em classes pequenas, coesas e testáveis.

2. Solução proposta

Tarifação	Pagamentos
Strategy com uma implementação por regra de cobrança. O valor base da diária é ajustado pela política vigente antes de compor o valor final do aluguel.	Strategy com uma implementação por meio de pagamento. A <code>Factory</code> recebe <code>FormaPagamento</code> e devolve o processador correspondente.
Nova regra de feriado ou evento: criar outra classe e registra-la no gerenciador.	Novo meio de pagamento: criar outro processador e registra-lo na <code>Factory</code> .

Fluxo do pagamento

- 1. O controller recebe `aluguelId` e `formaPagamento`.
- 2. `PagamentoService` busca o aluguel e le `getValorFinal()`.
- 3. `ProcessadorPagamentoFactory` seleciona a `Strategy`.
- 4. A `Strategy` valida e processa o valor.
- 5. O pagamento é salvo como `CONFIRMADO` e pode emitir recibo.

3. Justificativa dos padrões

Strategy

Strategy foi adotado nas duas funcionalidades porque existem famílias de algoritmos intercambiáveis: regras de ajuste tarifário e regras de processamento de pagamento. O consumidor conhece apenas a interface e não precisa saber qual cálculo concreto está sendo executado.

Factory

A Factory desacopla a criação do uso dos processadores. A tradução entre FormaPagamento e a classe concreta fica em um único ponto, eliminando condicionais nos controllers e serviços.

Singleton: GerenciadorTarifas

A política tarifária vigente é um recurso global. A previsão de diária (GET /quartos/{id}/diaria) e o valor final do aluguel devem aplicar a mesma regra no mesmo instante. Instâncias diferentes poderiam produzir preços divergentes para a mesma hospedagem.

Elemento	Implementação
Instância única	<code>private static GerenciadorTarifas instancia</code>
Construção controlada	<code>construtor privado</code>
Acesso global	<code>public static synchronized getInstance()</code>
Estado vigente	<code>volatile PoliticaTarifa politicaAtiva</code>

4. Benefícios da nova arquitetura

- Extensibilidade: novas tarifas e formas de pagamento entram sem alterar classes estáveis.
- Testabilidade: cada Strategy é validada isoladamente, e o serviço é testado com Mockito.
- Manutenção: regras de negócio ficam localizadas nas classes responsáveis.
- OCP: o sistema fica aberto para extensão e fechado para modificação.
- SRP: Factory seleciona, Strategy processa, Service orquestra e Repository persiste.

Diagrama oficial

O arquivo `diagrama_classes_sprint4.pdf` apresenta as realizações de interface, dependências e associações entre os dois conjuntos de padrões.

5. Demonstração - tarifação flexível

A execução abaixo foi realizada contra a aplicação Spring Boot com H2 em 2 de julho de 2026. Foi criado um quarto individual com diária base de R\$ 100,00, sem adicionais.

Captura 1 - consulta com a política PADRAO

```
PS> GET http://localhost:8080/quartos/1/diaria

HTTP 200 OK
{
  "quartoId": 1,
  "valorDiaria": 100.0
}
```

Captura 2 - ativacao da ALTA_TEMPORADA e nova consulta

```
PS> PUT http://localhost:8080/tarifas/ativa?politica=ALTA_TEMPORADA

HTTP 200 OK
{ "politicaAtiva": "ALTA_TEMPORADA" }

PS> GET http://localhost:8080/quartos/1/diaria

HTTP 200 OK
{
  "quartoId": 1,
  "valorDiaria": 130.0
}
```

Evidência

A mesma diária passou de R\$ 100,00 para R\$ 130,00, exatamente o acréscimo de 30% definido por TarifaAltaTemporada.

6. Demonstração - pagamento e recibo

Com a tarifa de alta temporada ativa, foi criado um aluguel de uma diária no valor final de R\$ 130,00. Em seguida, a API processou o pagamento via PIX.

Captura 3 - POST /pagamentos

```
PS> POST http://localhost:8080/pagamentos
Content-Type: application/json

{
  "aluguelId": 1,
  "formaPagamento": "PIX"
}

HTTP 201 Created
{
  "id": 1,
  "valor": 130.0,
  "formaPagamento": "PIX",
  "status": "CONFIRMADO",
  "aluguel": { "id": 1, "valorFinal": 130.0 }
}
```

Captura 4 - GET /pagamentos/1/recibo

```
HTTP 200 OK
=== RECIBO DE PAGAMENTO ===
Pagamento: 1
Data de emissao: 02/07/2026 02:14
Cliente: Maria Souza
Quarto: 1
Diarias: 1
Valor: R$ 130,00
Forma de pagamento: PIX
Status: CONFIRMADO
```

7. Verificação e qualidade

A verificação automatizada foi executada com o Maven Wrapper e JDK 17. A suite completa passou antes dos commits e novamente antes da entrega.

Metrica	Resultado
Testes executados	60
Falhas	0
Erros	0
Ignorados	0
Build	SUCCESS

Testes adicionados para pagamentos

- `ProcessadorPagamentoFactoryTest`: quatro formas corretas e rejeição de valor nulo.
- `ProcessadorPagamentoTest`: confirmação PIX e mínimo do cartão de crédito.
- `PagamentoServiceTest`: confirmação de aluguel existente e erro para aluguel inexistente.

Os arquivos TXT e XML produzidos pelo Surefire estão anexados ao repositório em docs/surefire-reports, preservando a evidência completa de cada classe de teste.

8. Conclusão

A Sprint 4 entrega duas famílias de regras extensíveis sem condicionais dispersas. Strategy permite trocar algoritmos, Factory concentra a seleção dos meios de pagamento e Singleton garante coerência global da tarifa. O resultado mantém as responsabilidades separadas e fornece uma base segura para novas políticas, novos meios e novas validações.

Status final

Implementação concluída, endpoints demonstrados e 60 testes aprovados sem falhas.